

Unit 3 · Topic Notes

Authentication — JWT Login System and OAuth 2.0 Basics

Course: Full Stack Application Development with MERN

Learning Objectives

1. Explain authentication vs. authorization
2. Build register, login and “who am I” endpoints with JWT
3. Protect routes with middleware and attach the token from React
4. Explain the OAuth 2.0 Authorization Code flow
5. Spot the security edge cases that make home-made auth unsafe

1. Authentication vs. Authorization

	Authentication (AuthN)	Authorization (AuthZ)
Question	Who are you?	What are you allowed to do?
Example	Logging in with email + password	Only admins can delete products
Failure code	401 Unauthorized	403 Forbidden
Covered in	This topic	Next topic (RBAC)

2. What Is a JWT?

HTTP is stateless: the server forgets you after every request. A **JSON Web Token** is a signed “ticket” the server hands you at login. You show it on every later request.

A JWT is three Base64URL parts joined by dots: `header.payload.signature`

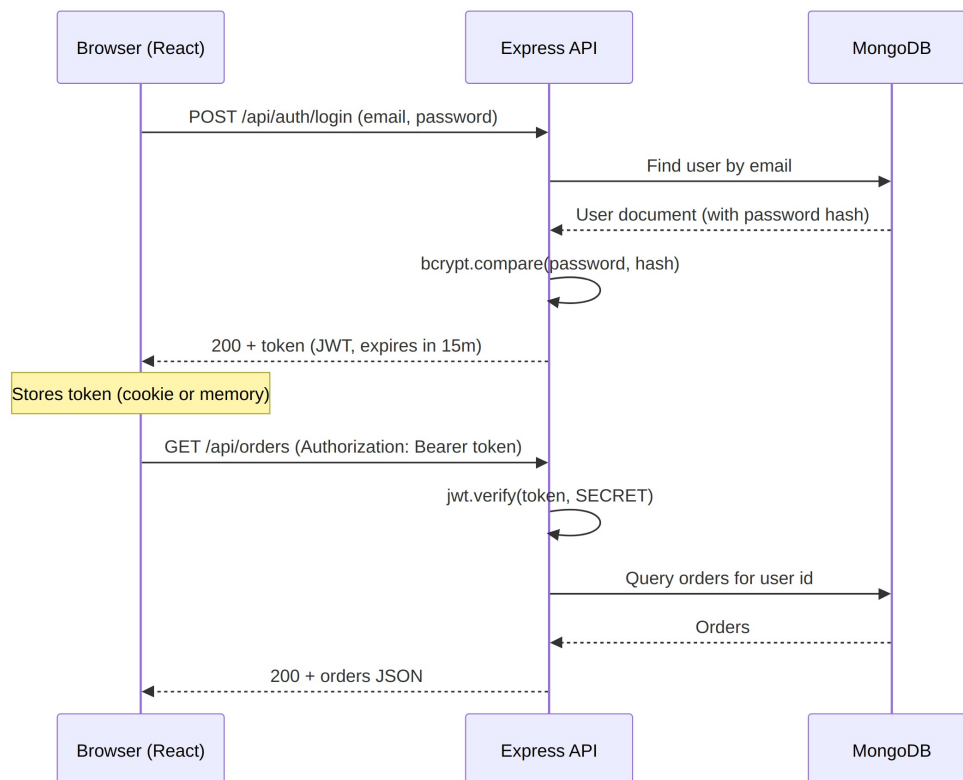
```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI2NGI3ZjBjMiIsImh0dCI6MTc1MDAwMDAwMCwiZXhwIjozNzUwMDAwOTAwfQ.Xk3...
```

Decoded payload:

```
{ "sub": "64b7f0c2a1b2c3d4e5f60789", "iat": 1750000000, "exp": 1750000900 }
```

- `sub` = subject (the user id), `iat` = issued at, `exp` = expiry (Unix seconds).
- The **signature** proves nobody changed the payload. Only the server knows the secret.

Critical point: a JWT is **signed, not encrypted**. Anyone can decode the payload (paste one into jwt.io). Never put passwords, card numbers or private data inside it.



JWT login sequence diagram

3. The User Model with Password Hashing

```

// models/User.js
import mongoose from "mongoose";
import bcrypt from "bcryptjs";

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true },
    email: { type: String, required: true, unique: true, lowercase: true, trim: true },
    password: { type: String, required: true, minlength: 8, select: false }, // hidden by default
    role: { type: String, enum: ["user", "admin"], default: "user" },
    passwordChangedAt: Date,
  },
  { timestamps: true }
);

// Hash only when the password field was set or changed
userSchema.pre("save", async function () {
  if (!this.isModified("password")) return;
  this.password = await bcrypt.hash(this.password, 12);
  if (!this.isNew) this.passwordChangedAt = new Date(Date.now() - 1000);
});

userSchema.methods.matchPassword = function (plain) {
  return bcrypt.compare(plain, this.password);
};

export default mongoose.model("User", userSchema);
  
```

`minlength: 8` is checked on the **plain** password because Mongoose runs validation before your `pre("save")` hook. (More on hashing in the last topic of this unit.)

4. Register and Login

```
// utils/generateToken.js
import jwt from "jsonwebtoken";

export const signToken = (userId) => {
  jwt.sign({ sub: userId }, process.env.JWT_SECRET, {
    expiresIn: process.env.JWT_EXPIRES_IN || "15m",
    algorithm: "HS256",
  });
};
```

```
# .env (generate a secret with: node -e "console.log(require('crypto').randomBytes(48).toString('hex'))")
JWT_SECRET=9f2c...long-random-string...
JWT_EXPIRES_IN=15m
```

```
// controllers/authController.js
import User from "../models/User.js";
import { signToken } from "../utils/generateToken.js";

const publicUser = (u) => ({ id: u._id, name: u.name, email: u.email, role: u.role });

export const register = async (req, res) => {
  const { name, email, password } = req.body ?? {}; // role is NOT accepted from the client
  const user = await User.create({ name, email, password });
  res.status(201).json({ token: signToken(user._id), user: publicUser(user) });
};

export const login = async (req, res) => {
  const { email, password } = req.body ?? {};
  if (typeof email !== "string" || typeof password !== "string") {
    return res.status(400).json({ message: "Email and password are required" });
  }

  const user = await User.findOne({ email: email.trim().toLowerCase() }).select("+password");
  const ok = user && (await user.matchPassword(password));

  // Same message whether the email or the password was wrong
  if (!ok) return res.status(401).json({ message: "Invalid email or password" });

  res.json({ token: signToken(user._id), user: publicUser(user) });
};
```

Try it:

```
curl -X POST http://localhost:5000/api/auth/register \
-H "Content-Type: application/json" \
-d '{"name":"Asha","email":"asha@example.com","password":"S3cure!pass"}'
```

5. The `protect` Middleware

```
// middleware/authMiddleware.js
import jwt from "jsonwebtoken";
import User from "../models/User.js";

export const protect = async (req, res, next) => {
  const [scheme, token] = (req.headers.authorization ?? "").split(" ");
  if (scheme !== "Bearer" || !token) {
    return res.status(401).json({ message: "Not authenticated" });
  }

  let payload;
  try {
    payload = jwt.verify(token, process.env.JWT_SECRET, { algorithms: ["HS256"] });
  } catch (err) {
    const message = err.name === "TokenExpiredError"
      ? "Session expired, please log in again"
      : "Invalid token";
    return res.status(401).json({ message });
  }

  const user = await User.findById(payload.sub);
  if (!user) return res.status(401).json({ message: "User no longer exists" });

  // Reject tokens issued before the last password change
  if (user.passwordChangedAt && payload.iat < Math.floor(user.passwordChangedAt.getTime() / 1000)) {
    return res.status(401).json({ message: "Password changed, please log in again" });
  }

  req.user = user; // available to every later handler
  next();
};
```

```
// routes/authRoutes.js
router.post("/register", register);
router.post("/login", login);
router.get("/me", protect, (req, res) => res.json({ user: publicUser(req.user) }));
```

6. Sending the Token from React

```
// src/services/api.js
import axios from "axios";

const api = axios.create({ baseURL: import.meta.env.VITE_API_URL });

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  (res) => res,
  (error) => {
    const isLoginCall = error.config?.url?.includes("/auth/login");
    if (error.response?.status === 401 && !isLoginCall) {
      localStorage.removeItem("token");
      window.location.assign("/login"); // session expired
    }
    return Promise.reject(error);
  }
);

export default api;
```

The `isLoginCall` check matters: a wrong password also returns `401`, and without the check the login page would keep reloading itself.

Where should the token live?

Storage	Pros	Cons
<code>localStorage</code>	Simple, works across domains	Any XSS bug can steal it
httpOnly cookie	JavaScript cannot read it, so XSS cannot steal it	Needs CORS + CSRF thinking

```
// Server: set the token as a cookie instead of returning it in JSON
res.cookie("token", token, {
  httpOnly: true,
  secure: process.env.NODE_ENV === "production", // HTTPS only in production
  sameSite: "lax", // "none" (+ secure) if frontend and API are on
different sites
  maxAge: 15 * 60 * 1000,
});
```

With cookies you also need `cors({ origin: CLIENT_URL, credentials: true })` on the server and `withCredentials: true` in Axios. In this course's labs, Bearer tokens are fine; know the trade-off for production.

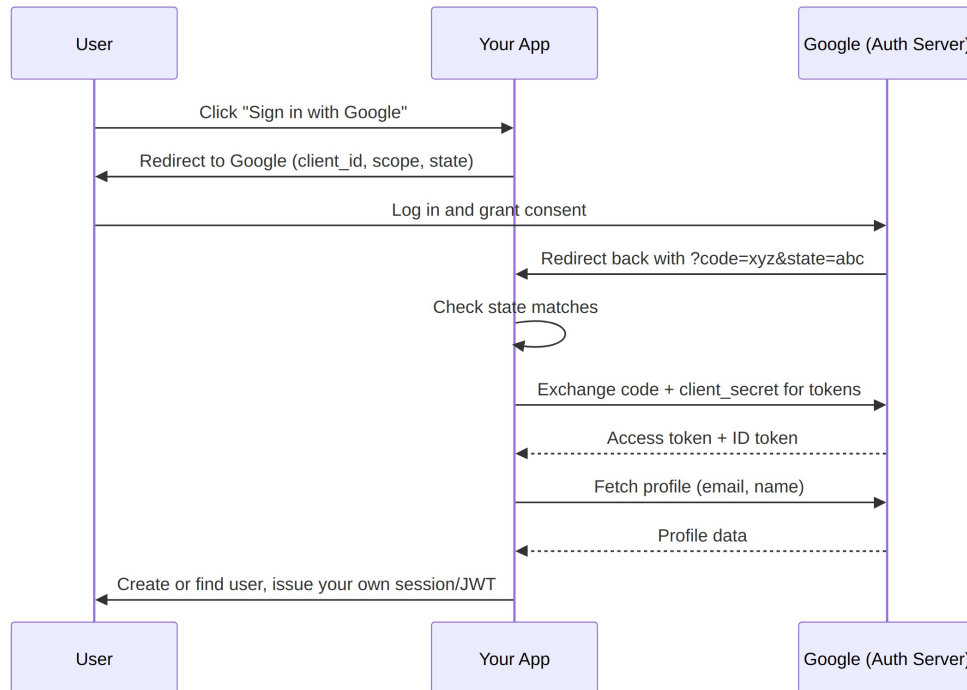
7. Edge Cases in JWT Authentication

- 1. Logout does not invalidate a JWT.** The server keeps no session, so a stolen token works until it expires. Mitigations: short expiry (15 minutes), a longer-lived **refresh token** stored in an httpOnly cookie, and (if needed) a server-side deny-list.
- 2. Weak or leaked secrets.** `JWT_SECRET=secret` can be brute-forced, and a committed `.env` exposes it. Anyone with the secret can forge a token for any user. Use 32+ random bytes, and rotate the secret if it leaks (this logs everyone out).
- 3. Always pin the algorithm.** `jwt.verify(token, secret, { algorithms: ["HS256"] })`. Historically, libraries were tricked by tokens claiming `alg: "none"`.
- 4. Different error, different reaction.** `TokenExpiredError` means "log in again"; `JsonWebTokenError` means tampering or garbage. Log the second kind.
- 5. The user changed after the token was issued.** Deleted user, changed password, demoted admin. That is why `protect` reloads the user from the database and compares `iat` with `passwordChangedAt`. Keeping the role **out of the token** and reading it from the database avoids stale-role bugs.
- 6. User enumeration.** Returning "email not found" vs "wrong password" tells attackers which emails are registered. Return one generic message. There is also a *timing* leak: when the email does not exist, no bcrypt comparison happens, so the response is faster. A common fix is to compare against a dummy hash anyway.
- 7. `select: false` does not apply to `create()`.** `User.create()` returns the document *including* the hash. That is why `publicUser()` builds the response explicitly. Never `res.json(user)`.
- 8. Email case and whitespace.** `Asha@example.com` and `asha@example.com` must be the same account. `lowercase: true` and `trim: true` in the schema, plus the same normalisation in `login`.
- 9. Simultaneous registration.** Two requests with the same email can both pass an "exists?" check. Rely on the unique index, and handle error `11000` as `409 Email already registered`.
- 10. Malformed headers.** Missing header, `Bearer` with no token, `Bearer null` (from `localStorage.getItem` returning the string `"null"`), lowercase `bearer`. The middleware must fail with `401`, never crash.
- 11. Never accept tokens in the URL** (`?token=...`). URLs end up in browser history, server logs and referrer headers.
- 12. No rate limit on login.** Attackers can try thousands of passwords. Add `express-rate-limit` (see the last topic).

8. OAuth 2.0 Basics ("Sign in with Google")

OAuth 2.0 lets a user give your app **limited access** to their account on another service **without sharing their password with you**.

Role	Who it is (Google sign-in example)
Resource owner	The user
Client	Your app
Authorization server	Google's login and consent screen
Resource server	Google's API that returns the profile



OAuth 2.0 Authorization Code flow

Key terms

- **Scope:** what you ask for (`profile` , `email`). Ask for the minimum.
- **Redirect URI:** where Google sends the user back. It must exactly match what you registered.
- **state :** a random value you send and later verify. It blocks cross-site request forgery on the callback.
- **Authorization code:** a short-lived, one-time code. It is swapped for tokens on the **server**, where the client secret stays hidden.
- **PKCE:** an extra proof for apps that cannot keep a secret (single-page and mobile apps).
- **OAuth vs. OpenID Connect (OIDC):** OAuth is about *authorization* (access). OIDC is a layer on top that adds an **ID token** and standard *authentication*. "Sign in with Google" is really OIDC.

8.1 Example with Passport

```
npm install passport passport-google-oauth2
```

```
// config/passport.js
import passport from "passport";
import { Strategy as GoogleStrategy } from "passport-google-oauth20";
import User from "../models/User.js";

passport.use(
  new GoogleStrategy(
    {
      clientID: process.env.GOOGLE_CLIENT_ID,
      clientSecret: process.env.GOOGLE_CLIENT_SECRET,
      callbackURL: `${process.env.API_URL}/api/auth/google/callback`,
    },
    async (accessToken, refreshToken, profile, done) => {
      try {
        const email = profile.emails?.[0]?.value?.toLowerCase();
        const verified = profile.emails?.[0]?.verified;
        let user = await User.findOne({ googleId: profile.id });

        if (!user && email && verified) {
          user = await User.findOne({ email }); // link an existing account
          if (user) { user.googleId = profile.id; await user.save(); }
        }
        if (!user) {
          user = await User.create({ name: profile.displayName, email, googleId: profile.id });
        }
        done(null, user);
      } catch (err) {
        done(err);
      }
    }
  )
);
```

```
// routes/authRoutes.js
router.get("/google", passport.authenticate("google", { scope: ["profile", "email"], session: false }));

router.get(
  "/google/callback",
  passport.authenticate("google", { session: false, failureRedirect: `${process.env.CLIENT_URL}/login?error=oauth` }),
  (req, res) => {
    // set our own token as an httpOnly cookie, then send the user back to React
    res.cookie("token", signToken(req.user._id), { httpOnly: true, sameSite: "lax", secure:
process.env.NODE_ENV === "production" });
    res.redirect(`${process.env.CLIENT_URL}/dashboard`);
  }
);
```

Note that a Google-only user has no password. Add `googleId: String` to the schema and make `password` required only when `googleId` is absent (`required: function () { return !this.googleId; }`).

8.2 OAuth edge cases

- **Redirect URI mismatch** is the most common setup error. `http://localhost:5000/...` and your deployed URL are two different URIs; register both in the Google Cloud console.
- **Account linking**: if `asha@gmail.com` registered with a password and later uses Google, linking blindly by email is only safe when the provider says the email is **verified**.
- **User denies consent**: Google redirects with `?error=access_denied`. Handle it; don't show a blank page.
- **Secrets**: the client secret belongs in `.env` on the server, never in React code.
- **Don't put your JWT in the redirect URL** (see edge case 11). Use a cookie, or a one-time code that React exchanges for a token.

9. Hands-On Lab (60 min, pairs)

1. Add the `User` model, `register`, `login` and `/me` routes from this note.

2. Register a user, log in, and call `/me` with and without the token (Postman: Authorization tab → Bearer).
 3. Break it on purpose and record the response for each: wrong password, unknown email (are the messages identical?), `Authorization: Bearer garbage`, an expired token (set `JWT_EXPIRES_IN=10s` and wait), registering the same email twice, registering with `"role": "admin"` in the body.
 4. Paste your token into jwt.io. What can you read? What does that tell you about what not to store?
 5. In React, create the Axios instance with interceptors and a login form that stores the token and redirects to a dashboard.
 6. **Stretch:** register a Google OAuth client and add “Sign in with Google”.
-

10. Quick Assessment

1. What is the difference between `401` and `403` ?
 2. Why can anyone read a JWT’s payload, and what does that mean for what you put in it?
 3. Why does `login` return the same message for a wrong email and a wrong password?
 4. A user logs out. Is their old token still valid? Why, and how do you reduce the risk?
 5. Why does `protect` re-load the user from the database instead of trusting the token alone?
 6. In the OAuth flow, why is the authorization code exchanged on the server and not in the browser?
 7. What is the purpose of the `state` parameter?
-

11. Common Mistakes to Avoid

- Storing plain-text passwords, or hashing with a fast hash like MD5/SHA-256
 - `res.json(user)` and leaking the password hash
 - A hard-coded or committed `JWT_SECRET`
 - Trusting `req.body.role` at registration
 - Tokens that never expire
 - Skipping the `isLoginCall` check and creating a redirect loop on `401`
-

Notes prepared for Full Stack Learning — MERN course, Unit 3.